Reliability vs Availability vs Durability

up is not the same as right

Watch the 3-min video

In one line: availability is being up, reliability is being correct over time, and durability is never losing acknowledged data; a system can be fully up and still wrong.

1 Three different questions

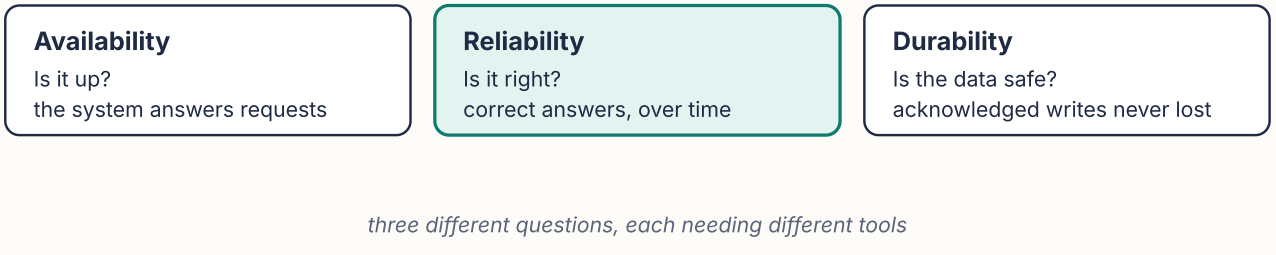


Figure 1. A reliable system must also be available, but an available system is not automatically reliable.

2 Up, but unreliable

Your banking app opens instantly and every request returns 200 OK, but the balance is wrong. The system is *available*, not *reliable*. Common ways this happens:

- Serving stale data from a broken or never-invalidated cache.
- Wrong prices after a bad deploy or config change.
- Silently dropping writes (a full queue, an ignored error).
- Returning 200 OK with an error message inside the body.



Figure 2. Uptime monitoring can be all green while users get wrong answers.

3 Measuring reliability

Not just “is it up?” but “is it right?”:

- **Error rates** and failed requests, including errors hidden in 200 responses.
- **Time between failures** (MTBF) for correctness incidents, not only outages.
- **Correctness checks**: reconcile totals (orders vs payments), validate responses end to end, run synthetic transactions.

4 Durability: does the data survive?

Durability belongs to storage: once a write is acknowledged, it must never be lost. It's not about reaching the data right now; it's about the data still existing. You get it with replication across disks and zones, checksums to detect corruption, and backups you actually restore from in tests.

AMAZON S3: DIFFERENT TARGETS FOR DIFFERENT PROPERTIES

S3 Standard is designed for **99.999999999% durability** (eleven nines): store 10 million objects and you might lose one about once every 10,000 years. But it's designed for **99.99% availability**: occasionally you can't reach a file, yet it's almost certainly still there.

5 Tools for each property

Property	Main tools
Availability	Redundancy, health checks, fast failover, removing single points of failure
Reliability	Testing, input validation, idempotent operations, safe deploys (canaries), monitoring correctness, not just uptime
Durability	Replication across zones, write-ahead logs, checksums, backups, tested restores

COMMON MISTAKES

- Treating 99.99% uptime as “reliable”.
- Backups that were never restored in a test.
- Acknowledging a write before it's safely stored.
- Monitoring only process health, not response correctness.

6 Key terms

Availability: the system answers

Durability: acknowledged data survives

Checksum: detects corrupted data

Reliability: the answers are correct, over time

Idempotent: safe to retry

Restore test: proving backups actually work

7 Interview questions

Q1. Can a system be highly available but unreliable? BASIC

Yes. It can be up and fast while returning wrong or stale data, dropping writes or hiding errors inside 200 responses.

Q2. What is durability? BASIC

The guarantee that acknowledged data is never lost, achieved with replication, checksums and tested backups.

Q3. How is durability different from availability? BASIC

Availability: can I reach the data now? Durability: does the data still exist? S3 has 11 nines of durability but 4 nines of availability.

Q4. How would you detect that an e-commerce site is up but wrong? SCENARIO

Reconcile orders against payments and inventory, run synthetic purchases end to end, alert on business metrics (e.g. orders per minute dropping), and track errors inside responses.

Q5. Design storage so an acknowledged order is never lost. SCENARIO
Write to a database that replicates synchronously to another zone (or a quorum) before acknowledging, keep a write-ahead log, take regular backups and restore-test them.

Q6. A retry charged a customer twice. Which property failed, and how do you fix it? SCENARIO
Reliability (correctness). Make the payment API idempotent with an idempotency key, so a retried request returns the first result instead of charging again.

Q7. Do backups give durability? TRICKY
Only if they're recent, stored elsewhere, uncorrupted and restorable. Untested backups often fail when needed; replication alone doesn't protect against accidental deletes or bad writes.

Q8. Is replication enough for durability? TRICKY
It protects against losing a disk or a zone, but it also copies mistakes instantly: a bad delete or corruption replicates too. You still need point-in-time backups.

Next: SD-A-012 · Scalability backendroadmap.com